

บทที่ 2

ทฤษฎีพื้นฐานที่ใช้ในโครงการ

ในหัวข้อนี้จะกล่าวถึงทฤษฎีพื้นฐานที่สำคัญ ในส่วนที่เกี่ยวข้องกับ โครงการนี้ ได้แก่ ทฤษฎีเกี่ยวกับคอมโพเนนต์ , ทฤษฎีเกี่ยวกับ .Net Framework , ทฤษฎีเกี่ยวกับ .Net Assembly และ ทฤษฎีการส่งข้อมูลผ่านเครือข่ายแบบ Multicast

2.1 คอมโพเนนต์

มีความตกลงทั่วไปในสังคมของผู้พัฒนาซอฟต์แวร์ โดยให้ความหมายของคอมโพเนนต์ คือ คอมโพเนนต์คือหน่วยอิสระที่สามารถประกอบขึ้นจากคอมโพเนนต์อื่นๆ เพื่อที่จะสร้างระบบซอฟต์แวร์ แต่ก็ได้มีบุคคลอื่นๆนำเสนอนิยามอื่นๆ ของคอมโพเนนต์

Council and Heineman ให้นิยามความหมายของ คอมโพเนนต์ ดังนี้

“A software element that conforms to a component model and can be independently deployed and composed without modification according to a composition standard.”

Ian Sommerville ได้ให้ความหมายของคอมโพเนนต์ไว้ดังนี้

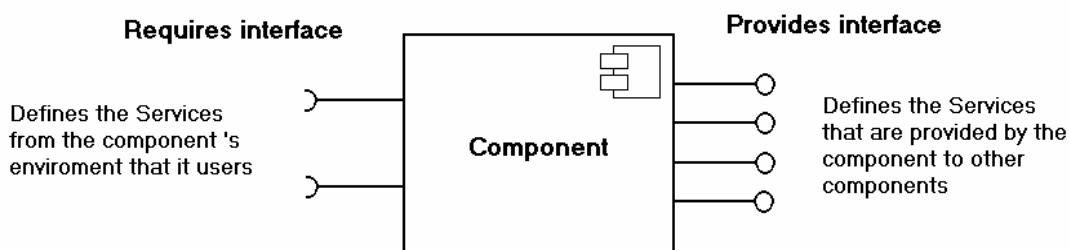
“A component is a software unit whose functionality and dependencies are completely defined by a set of public interfaces. Component can be combined with other components without reference to their implementation and can be deployed as an executable unit.”

2.1.1 คุณสมบัติของคอมโพเนนต์

คุณสมบัติโดยทั่วไปที่สำคัญ ของคอมโพเนนต์มี 5 ประการ ดังนี้

1. Standardised หมายความว่าคอมโพเนนต์ที่นำมาใช้ในกระบวนการพัฒนาซอฟต์แวร์จะต้อง ถูกต้อง ตามข้อกำหนดของคอมโพเนนต์โมเดล โดยคอมโพเนนต์โมเดลนี้ จะกำหนด ข้อกำหนดต่างๆ เช่น คอมโพเนนต์อินเตอร์เฟซ,คอมโพเนนต์เมตาเดต้า,คอมโพเนนต์คอมโพสิชัน,คอมโพเนนต์ค็อกกิวเมนต์ชั่น ,คอมโพเนนต์ดีพลอยเมนต์
2. Independent หมายความว่าคอมโพเนนต์ควรเป็นอิสระ และเป็นไปได้ที่จะประกอบและดีพลอย มันโดยปราศจากการใช้คอมโพเนนต์อื่นที่ระบุเฉพาะเจาะจง ในสถานะการที่คอมโพเนนต์ต้องการบริการจากคอมโพเนนต์อื่น จะต้องกำหนด “requires interface” อย่างชัดเจน

3. Composable สำหรับคอมโพเนนต์ที่ composable การติดต่อกับภายนอกทั้งหมดจะต้องผ่าน “publicly defined interface” ซึ่งอินเตอร์เฟซเหล่านั้นจะภายนอกสามารถเข้าถึงข้อมูลภายในของคอมโพเนนต์เช่น method และ attribute ต่างๆ ได้
4. Deployable คอมโพเนนต์เป็นไปนาริสามารถนำไปใช้งานในระบบนั้นได้โดยไม่ต้องคอมไพล์ระบบนั้นและ คอมโพเนนต์นั้นใหม่
5. Documented คอมโพเนนต์ควรจะต้องมีเอกสารอธิบายเป็นอย่างดีเนื่องจากคอมโพเนนต์เป็น blackbox ซึ่งผู้ใช้จะไม่สามารถรู้รายละเอียดภายในได้ โดยสิ่งที่สำคัญที่เอกสารจะต้องอธิบายคือข้อกำหนดของทุกๆ public interface ทั้งหมด ยิ่งไปกว่านั้น ผู้ใช้ที่จะตัดสินใจว่าจะเลือกใช้คอมโพเนนต์นี้หรือไม่สามารถตัดสินใจได้ว่าคอมโพเนนต์นี้ตรงกับความต้องการหรือไม่



รูปที่ 2.1 แสดงส่วนประกอบของคอมโพเนนต์

2.1.2 คอมโพเนนต์โมเดล

คอมโพเนนต์โมเดลเป็นการนิยามมาตรฐานสำหรับการพัฒนาคอมโพเนนต์, ดอกคิวเมเตชั่น, คีพลอยเมนต์ มาตรฐานเหล่านี้เป็นมาตรฐานสำหรับผู้พัฒนาคอมโพเนนต์ เพื่อจะได้แน่ใจได้ว่าคอมโพเนนต์ทั้งหลายสามารถทำงานร่วมกันได้ ยิ่งไปกว่านั้นมาตรฐานเหล่านี้ยังได้ให้โครงสร้างพื้นฐานในการสำหรับการ เอกซิกิวชั่น และได้ให้ มิดเดิลแวร์ ที่สนับสนุน โอเปอเรชั่นต่างๆของ คอมโพเนนต์

2.1.3 การพัฒนาซอฟต์แวร์เชิงคอมโพเนนต์สำหรับการนำกลับมาใช้ใหม่

ข้อดีที่สำคัญสำหรับการพัฒนาที่สุดสำหรับการพัฒนาซอฟต์แวร์เชิงคอมโพเนนต์ คือความสามารถในการนำกลับมาใช้ใหม่(Reuse) โดยปกติแล้วคอมโพเนนต์มีความสามารถในการนำกลับมาใช้ใหม่ได้ในตัวเองอยู่แล้ว แต่ก็มีหลักการที่สามารถเพิ่มความสามารถในการนำกลับมาใช้ใหม่ให้กับคอมโพเนนต์ ดังนี้

1. ไม่ออกแบบเมทอด ให้เจาะจงสำหรับแอปพลิเคชันใดแอปพลิเคชันหนึ่ง
2. ต้องมีเมทอดให้ครบสมบูรณ์ ตามฟังก์ชันการทำงานของ คอมโพเนนต์

3.1 เปลี่ยนชื่อต่างๆให้เหมาะสมและ เจอเนอเรชันมากขึ้น

4. เพิ่ม “configuration interface” ทำให้คอมโพเนนต์ สามารถปรับเปลี่ยนได้ ตามสถานการณ์ เมื่อนำคอมโพเนนต์ไปใช้งาน

2.1.4 ปัญหาสำหรับการพัฒนาสำหรับการพัฒนาซอฟต์แวร์เชิงคอมโพเนนต์

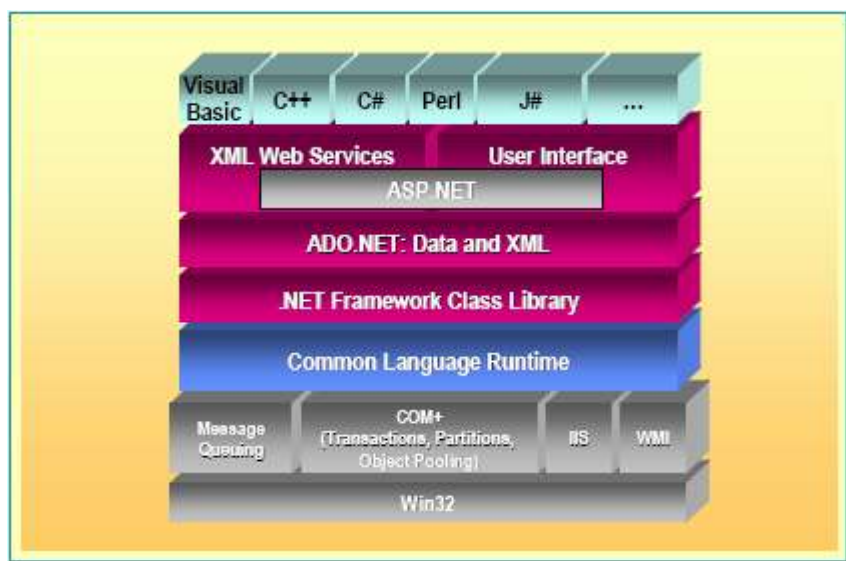
เนื่องจากคอมโพเนนต์เป็นเสมือน blackbox ผู้พัฒนาที่นำคอมโพเนนต์ไปใช้งานนั้น ผู้ใช้จะไม่สามารถรู้รายละเอียดภายในได้เลยว่าคอมโพเนนต์ทำงานอะไรบาง ดังนั้น จึงมีคำถามว่าผู้ใช้จะรู้ได้อย่างไรว่าคอมโพเนนต์ที่นำมาใช้นั้นมีความน่าเชื่อถือมากน้อยแค่ไหน ซึ่งส่งผลต่อซีเคียวริตี้คอมโพเนนต์ ที่ไม่น่าเชื่อถืออาจนำมาซึ่งปัญหาต่างๆ เช่น โค้ดที่ผิดพลาดภายในคอมโพเนนต์ อาจทำให้ระบบล่มและ โค้ดที่ไม่หวังทำให้ ซีเคียวริตี้ในระบบมีปัญหา เช่น สามารถเกิด Trojan horse ได้

2.2 .NET Framework

.NET Framework คือ โครงสร้างพื้นฐานในการพัฒนาโปรแกรมคอมพิวเตอร์ที่ไม่โครซอฟต์ พยายามรวมระบบปฏิบัติการ และอินเตอร์เน็ตให้เป็น แพลตฟอร์มเดียวกัน ถูกออกแบบมา เพื่อให้อำนวยความสะดวกในการพัฒนาโปรแกรมสมัยใหม่ ที่ใช้งานในระบบเครือข่าย

2.2.1 ส่วนประกอบของ .Net Framework

.Net Framework มีส่วนประกอบ หลัก ๆ ดังนี้



รูปที่ 2.2 แสดง ส่วนประกอบของ .NET Framework

2.2.1.1 Common Language Runtime (CLR)

เป็นส่วนพื้นฐานที่ติดต่อกับระบบปฏิบัติการ Windows ทำหน้าที่เป็น run-time environment ให้กับโปรแกรมที่เขียนขึ้นสำหรับใช้บน .Net CLR มีส่วนของ compiler ทั้งที่เป็นแบบปกติ (compile ก่อนที่จะนำโปรแกรมไปใช้) และแบบ Just-In-Time (compile เมื่อจะใช้โปรแกรมนั้น ๆ) มีส่วนของ Memory Management ที่เอาไว้สำหรับจัดสรรหน่วยความจำของเครื่องให้กับ โปรแกรม รวมไปถึงการทำ Garbage Collection (การเรียกคืนหน่วยความจำที่ไม่ได้ใช้อีกต่อไป) ส่วนของ Common Type Systems (CTS) ทำให้ภาษาต่าง ๆ ที่เขียนขึ้นบน .Net สามารถทำงานร่วมกันได้ เพราะขนาด และรูปแบบของข้อมูลที่เก็บไว้นั้นเป็นรูปแบบเดียวกัน

2.2.1.2 NET Framework Class Library

เป็น class library พื้นฐาน ที่โปรแกรมต่าง ๆ ไม่ว่าจะเขียนด้วยภาษาใดบน .Net ก็สามารถใช้ class library นี้ร่วมกันได้ เช่น การติดต่อระบบฐานข้อมูล (ADO.Net), การติดต่อกับ file system ของ server (IO), ฯลฯ

2.2.1.3 Programming Languages

เป็นเซตของ ภาษาคอมพิวเตอร์ ที่ถูกออกแบบมาเพื่อการเขียนโปรแกรมบน .Net Framework ไม่ใคร่ขอพจน์นั้นเน้นไปที่ 3-4 ภาษาหลัก ๆ ได้แก่ VB.Net ซึ่งเป็นตัวที่พัฒนาต่อมาจาก VB, C# ซึ่งเป็นภาษาใหม่ที่มี syntax ใกล้เคียงกับ Java และ C++, Visual C++, และ JScript.Net ส่วนภาษาอื่น ๆ นั้น มีบริษัท หรือหน่วยงานอื่น ๆ เป็นผู้พัฒนาขึ้น ซึ่งคาดว่าจะมีเป็นสิบ ๆ ภาษา สำหรับ .Net Framework นั้น ไม่ว่าจะเขียนโปรแกรมด้วยภาษาใดก็ตาม Compiler ใน CLR ก็จะ compile โปรแกรมนั้นให้อยู่ในรูปของ Intermediate Language (IL) ซึ่งจะถูกนำไปแปลเป็นภาษาเครื่อง (Native Code) อีกทีเมื่อตอนที่นำไปใช้

2.2.1.4 ASP.Net

เป็น server side script ที่พัฒนาต่อมาจาก ASP ตัวเก่า เพื่อให้โปรแกรมเมอร์สามารถพัฒนา web application ให้อยู่บน .Net ได้สะดวกขึ้น ASP.Net นี้ถึงแม้จะอ้างอิงมาจาก ASP ตัวเก่า แต่ก็มี syntax หลายส่วนที่เปลี่ยนแปลงไป

2.2.2 ข้อดีของการพัฒนาซอฟต์แวร์บน .Net Framework

การพัฒนาซอฟต์แวร์บน .Net Framework มีข้อดีต่างๆดังนี้

2.2.2.1 Executable code สามารถเรียกใช้เกือบทุก OS และเป็นอิสระต่อ Hardware

Microsoft ทำได้อย่างไร เป็นเรื่องที่น่าสนใจ เรื่องนี้ Microsoft ต้อง Install Program ลงไปชุดหนึ่งครับ ที่เรียกว่า Microsoft .Net Framework โปรแกรมตัวนี้จะจำลองตัวเองเป็น OS เล็กๆ ที่หน้าตาเหมือนกับ DOS, และเราสามารถ Run โปรแกรมอะไรก็ได้ที่เขียนขึ้นมาสำหรับ .Net Framework ที่ผมพูดว่า Linux และ Solaris สามารถใช้ Microsoft Office.Net ได้นั้น มันยังไม่จริงนะครับ เพราะวันนี้ทาง Microsoft ยังไม่ Implement .Net Platform บน Linux หรือ Solaris แต่ก็มีบางบริษัท ประกาศว่าจะ Implement ส่วนนี้ให้ ขอเพียงแค่ Microsoft ว่าจ้างเท่านั้น ส่วนตัว program ที่ทำงานบน .Net Framework นั้น เป็น .exe ที่สามารถ run ได้โดยตรงจาก OS แต่ code นั้นไม่ใช่เป็น code ของ Intel Pentium หรือ CPU ใดๆ แต่หากเป็น Machine code ที่เป็นภาษาที่ Microsoft คิดค้นขึ้นครับ เมื่อเวลา Run Code นี้จะถูก Compile ให้เป็น Machine Code แท้ๆ ของเครื่องที่ใช้งานอยู่ ด้วยวิธีนี้ เราสามารถ run .exe เดียวกันในบนทุกเครื่อง

2.2.2.2 Library เป็นชุดเดียวกัน ไม่ว่าจะเป็นภาษาคอมไพเลอร์ภาษาใดก็ตาม และมีความสมบูรณ์มาก

ภาษาต่างๆ มี Library ไม่เหมือนกัน ทำให้การเรียนรู้แต่ละภาษานั้นกินเวลามาก และถ้าเขียน module ที่ใช้ภาษาที่มีความสามารถสูง เช่นภาษา C ที่สามารถรองรับ จำนวน parameters ที่ผ่านเข้าหา function ไม่อันเช่น printf(“%d%d%d”, 1, 2, 3); พอใช้ภาษา Pascal ไปอ่าน library ตัวนี้จะไม่รองรับครับ ทำให้การเชื่อมระหว่างภาษาเป็นไปได้โดยยาก Microsoft เลยตัดสินใจที่ยกเลิก Library ของทุกภาษา และออกแบบ Library ตัวใหม่ให้ทุกภาษาใช้ร่วมกัน ต่อไปทุกภาษาจะมี Library ชุดที่สมบูรณ์ที่สุดใช้ ใน Library ชุดนี้มีเกือบทุกสิ่งที่ Programmer อยากรจะมี ADO, Threading ,Networking เป็นต้น

2.2.2.3 มีการควบคุมสิ่งแวดล้อมในการ Run โปรแกรมเป็นอย่างดี

จุดขายของ Java อยู่ที่ โปรแกรมที่เป็น .class นั้น เมื่อนำมาทำงานจะถูกควบคุมไม่ให้ทำงานออกนอกกรอบนอกทางไปรวนระบบ ซึ่งคุณสมบัตินี้ถูก Microsoft นำมาเลียนแบบเป็นที่ มีการพิสูจน์ที่มาของ Module โดยใช้การเข้ารหัสมาช่วย ผู้ใช้จะมีสิทธิเลือกเองว่าเชื่อถือ Module นั้นหรือไม่ ใน Platform นี้ยังสามารถกำหนดสิทธิผู้ใช้ตาม OS ได้ มีการจัดการ Garbage Collection ได้ ทำให้ไม่เกิด Memory Leak อีกด้วย

2.2.2.4 สามารถใช้ภาษาใหม่คือ C# ที่รวมเอาข้อดีของ C, C++, Java, Delphi และ Ruby เข้าด้วยกัน

Microsoft เคยเป็นผู้สนับสนุน Java แต่ Microsoft ก็ไม่สามารถยอมรับความซ้ำของ Java ได้ ดังนั้น ใน Visual J++ จึงทำการปรับปรุง Java โดยเพิ่ม keyword ที่ชื่อว่า Delegate ด้วย keyword ตัวนี้ทำให้ Java สามารถเข้าถึง Windows API และ COM components ได้โดยตรง ทำให้เร็วกว่า Java ของ Sun 5 เท่า แต่เสียความ Compatible กับ Java เจ้าอื่น ทำให้ Sun ตัดสินใจฟ้อง Microsoft และ Microsoft ก็แพ้คดี หลังจากนั้น Microsoft ประกาศเลิก สนับสนุน Java Web ของ Microsoft ทั้งหมดตัดเอา Java ที่จี้ เมื่อเวลาผ่านไป Microsoft Visual Basic ก็ได้รับความนิยมสูงขึ้น แต่ก็ยังมีโปรแกรมเมอร์อยู่เป็นจำนวนมากที่ยอมรับความง่ายเกินไปของ Visual Basic ไม่ได้ เมื่อเรื้อแตกจากภาษา C และ C++ จึงหันไปหา Java ทำให้สังคม Java คึกคักมาก ทาง Microsoft ก็รู้ถึงปัญหานี้ จึงหาทางออกแบบภาษาใหม่ ให้ออกแนวภาษา C และ C++ ทำให้ programmer ต่างๆ ผู้ใช้ Java อยู่ สามารถเรียนรู้ภาษานี้ได้โดยไม่ยาก เพื่อการนี้ Microsoft ทำการซื้อตัว Anders Hejlsberg ผู้เป็นผู้ออกแบบ Turbo Pascal และ Delphi จาก Borland มาออกแบบภาษาใหม่โดยมีรหัสว่า Cool และในที่สุดได้ชื่อเป็นทางการว่า C#

ภาษานี้เป็นภาษาที่รวมเอาความสามารถดีๆ จาก C, C++, Java, และ Delphi เข้าด้วยกันใน C# version 1.0 และรวมความสามารถบางอย่างของ ภาษา Ruby เข้ามาอีกใน C# version 2.0 C# มีความคล้าย Java อยู่ถึง 80% มันเป็นภาษาที่ใช้แนวคิดของ OOP อย่างเต็มตัว และใช้ความสามารถของ .Net ได้เกือบ 100%

2.3 .Net Assembly

Assembly เป็นศัพท์เฉพาะที่ Microsoft สร้าง Assembly ตามความหมายของ .NET นั่นก็คือพวก file .dll รุ่นใหม่นั้นเอง ในอดีตจนถึงปัจจุบันการใช้ file .dll นั้นมีปัญหาหลายอย่าง จึงเป็นเหตุผลว่าทำไมต้องมี Assembly

2.3.1 ประวัติ ความเป็นมาและความหมายของ .Net Assembly

ในยุคของ DOS นั้นโปรแกรมยังไม่ใหญ่มาก โปรแกรมส่วนใหญ่ก็จะมี Graphic มีแต่ตัวหนังสือ (ยกเว้นเกมส์) การแสดงตัวอักษรต่างๆ บนหน้าจอหรือการติดต่อกับอุปกรณ์ต่างๆ เช่น Harddisk, floppy disk นั้นบน DOS และ BIOS ได้เตรียมฟังก์ชันการใช้งานให้เราสามารถเรียกใช้งานได้อยู่แล้ว และถึงแม้ว่าเราจะไม่ใช่ฟังก์ชันดังกล่าว เราเขียนติดต่อกับ I/O โดยตรงก็ทำได้โดยง่าย และขนาดของโปรแกรมไม่ใหญ่ ถ้าเรามองมุมมองของการเขียนโปรแกรม โปรแกรมที่เป็น .exe ส่วนมากบน DOS นั้นสร้างมาจากภาษา C ซึ่งแน่นอนครับว่าเกือบทุกโปรแกรมต้องมี

การเรียกใช้ฟังก์ชันที่ชื่อว่า printf() และถ้าเรามีโปรแกรม .exe อยู่ 1,000 ตัวนั้นก็หมายความว่าเรามี code ของ printf() แทรกอยู่ทั้ง 1,000 โปรแกรม

มาถึง Windows 16 bits Windows รุ่นเปลี่ยนแปลงโลก นั่นก็คือ Windows 3.1 ซึ่ง Windows นั้นเป็น OS ที่ใช้ Graphic User Interface ซึ่งเลียนแบบมาจาก Apple McIntosh สิ่งที่เปลี่ยนไปก็คือ โปรแกรมต่างๆ นั้นแสดงผลเป็น Graphic ทั้งหมด เรามามองถึงมุมมองคนออกแบบ Windows รุ่นแรกๆ กันหน่อยดีกว่า ในยุคนั้น Harddisk มีขนาดเล็กมาก Memory ก็น้อย ความเร็วของ CPU ก็จำกัด ทุกสิ่งทุกอย่างดูเหมือนว่ายังไม่พร้อมสำหรับ Windows ครับ ดังนั้นการออกแบบ Windows ในยุคนั้นจึงถือเอาหลักประหยัด Resource ให้มากที่สุดเท่าที่จะเป็นไปได้ ดังนั้นสิ่งที่ Microsoft นำเอามาใช้ในเวลานั้นก็คือสิ่งที่เรียกว่า Shared Library คือการที่เอา Library ที่มีการใช้ร่วมกัน (เช่น printf()) เอาไว้ในแฟ้มข้อมูลแยกต่างหาก จะไม่รวมเข้าไปอยู่ใน .exe ดังนั้นถ้าโปรแกรม 1,000 ตัว มีการเรียกใช้ printf() จะไม่มีโปรแกรมตัวไหนเลยที่เก็บ code ของ printf() เวลารันโปรแกรม จะมีการดึงเอา code ของ printf() มาใช้ทำให้โปรแกรมแต่ละตัวมีขนาดเล็กลงมากจนน่าแปลกใจ โปรแกรมที่พิมพ์คำว่า Hello, World รุ่นบน DOS และบน Windows นั้นมีขนาดแทบไม่แตกต่างกัน ถ้าไม่มีเรื่อง Shared Library มาช่วย โปรแกรม Hello, World บน Windows อาจจะใหญ่กว่ารุ่นบน DOS เป็นหลายร้อยเท่าครับ

และจุดที่มีการประชาสัมพันธ์กันค่อนข้างจะมากสำหรับ Shared Library ก็คือความดีที่สามารถแก้บั๊กได้ เช่นถ้า printf() มีบั๊ก เราไม่จำเป็นต้องหาทั้ง 1,000 โปรแกรมมาใหม่ ในเมื่อโปรแกรมแต่ละตัวเรียกใช้ printf() ข้างนอกเราเพียงปรับปรุง Shared Library ของ printf() เท่านั้น โปรแกรมทุกตัวก็ถือว่ามีการแก้บั๊กแล้วทั้งหมด และ Shared Library ที่กล่าวมา จะอยู่ในแฟ้มข้อมูลที่เรารู้จักกันดี นั่นก็คือ .dll นั่นเอง (Dynamic Linked Library)

มายุค Windows 95 .dll ซึ่งเคยเป็นพระเอกนั้น ถูกมองเป็นผู้ร้ายไป จนกระทั่งทุกวันนี้ มีผู้คนเรียก .dll กันติดปากว่า dll Hell เนื่องจาก ปัญหาหลักที่เกิดขึ้นกับ .dll นั่นก็คือมันเป็นสิ่งที่ใช้ร่วมกันกันหลายโปรแกรม ทำให้เกิดปัญหาดังนี้ครับ

การตั้งชื่อ .dll เป็นได้นะครับว่าบริษัทต่างๆ ที่สร้าง .dll ขึ้นมานั้นอาจจะสร้างชื่อเดียวกัน ดังนั้นเมื่อเวลาเราลง โปรแกรม จะทำให้มีการทับกันเกิดขึ้น

ถึงแม้ว่า .dll จะมาจากบริษัทเดียวกัน แต่ก็อาจจะเป็นคนละรุ่น และบ่อยครั้งครับที่เมื่อเรา .dll รุ่นใหม่ไปทับ .dll รุ่นเก่า มันจะทำให้โปรแกรมที่ install ไว้ก่อนหน้านี้ ทำงานไม่ได้ ซึ่งทุกวันนี้ยังมีปัญหานี้อยู่

ทาง Microsoft จึงได้ออกแบบ .dll แบบใหม่ขึ้น รู้จักกันในชื่อ COM Component มันเป็นความพยายามที่จะแก้ปัญหา dll Hell โดยที่ COM Component นั้นสามารถรองรับ Interface Inheritance ทำให้ .dll ตัวเดียวกันสามารถมีหลาย Version แต่ใช่ว่ามันจะเป็นยาวิเศษที่สามารถ

รักษาโรคภัยตัวนี้ได้ 100% มีคำถามเกิดขึ้นครับว่า มีซักกี่บริษัทที่เวลาออก COM Component รุ่นใหม่ จะกลับไปทดสอบดูว่า COM Component รุ่นนี้เข้ากันได้กับโปรแกรมรุ่นเก่าโดยสมบูรณ์ เชื่อว่าแทบจะไม่มีบริษัทไหนเลยที่ลงทุนลงแรงขนาดนั้น แม้แต่ Microsoft เองก็ตาม ดังนั้นปัญหาการลงโปรแกรมตัวหนึ่ง ทำให้โปรแกรมตัวอื่นที่เคยทำงานได้กลับทำงานไม่ได้ นั่นยังคงเกิดขึ้นอยู่แม้ในยุคปัจจุบันครับ

ปัญหา DLL Hell เริ่มเลวร้ายขึ้น เพราะจำนวนโปรแกรมที่มีการพัฒนาขึ้นสำหรับ Windows นั้นมีปริมาณเพิ่มขึ้นแบบทวีคูณ ทำให้ Windows นั้นถือว่าเป็น OS ที่ไม่ Stable เมื่อเทียบกับค่าย UNIX เช่น Linux เป็นต้น แต่ถ้าไล่กันเข้าไปจริงๆ ปัญหาที่เกิดจาก DLL Hell นั้นเอง

Microsoft ไม่นิ่งนอนใจรับอุตสาหกรรมที่ลงแรงมากที่สุดเท่าที่เคยมีมา แต่กลับไปตายนั่นแบบนี้ ทาง Microsoft จึงประกาศโครงการที่ชื่อว่า Fusion Technology เพื่อแก้ไขปัญหาด DLL Hell โดยเฉพาะครับ และ Windows 98SE นั้นถือว่าเป็นรุ่นแรกของ Technology นี้ครับ Microsoft ทำดังนี้ครับ

เมื่อเวลา Install Programming นั้น ถ้าพบว่า .DLL ที่กำลัง Install นั้นชนกับ DLL ที่มีอยู่แล้ว ทาง Windows จะแจ้งโปรแกรมที่กำลัง Install ว่าได้ย้าย .DLL ที่กำลัง Install นั้นไปอยู่ที่อื่น ทำให้ไม่ต้องทับกัน ซึ่งการย้ายแบบนี้เราเรียกว่า Side-by-Side และ Technology ที่ใช้ในรุ่นนี้มี Code Name ว่า Hydrogen

แต่มันก็แทบไม่ได้ช่วยอะไรครับ เพราะโปรแกรมที่กำลัง Install นั้นต้องเรียกใช้ API พิเศษ ซึ่งมีเพียงโปรแกรมไม่กี่ตัวครับที่ใช้ประโยชน์จากมัน

Fusion Technology รุ่นต่อมาคือ Windows ME / 2000 ซึ่งมีสิ่งที่เรียกว่า Windows File Protection (WFP) หรือ System File Protection (SFP) ซึ่งทำการป้องกันไม่ให้ System Files ที่สำคัญถูกโปรแกรมใดๆ ทับ ถ้าถูกทับจริงๆ ก็จะมีกรรมวิธีในการ Restore เพื่อรักษาให้ระบบเสถียร แต่นั่นก็สามารถป้องกันได้เฉพาะ System Files ไม่รวมถึง Application ทั่วไป

เมื่อมาถึง .NET ซึ่งเป็นยุคที่กำลังเครื่องของคอมพิวเตอร์และ Resource ต่างๆ เริ่มมีมากขึ้น Harddisk ขนาด 20 GB ที่ใช้ตามบ้านถือว่าขนาดเล็ก ซึ่งถ้าเปรียบเทียบกับก่อนหน้านี้ประมาณ 10 ปี ขนาด Harddisk 20GB นั้นถือว่าเป็นศูนย์คอมพิวเตอร์ขนาดยักษ์เลยทีเดียว

เมื่อกำลังเครื่องของคอมพิวเตอร์ไม่ใช่เป็นข้อจำกัด ดังนั้นแนวทางของ Fusion Technology นั้นจึงออกไปในแนวทางที่ไม่มีการทับกัน รุ่นต่างๆ ให้ถือว่าเป็นคนละเพิ่มข้อมูลกัน เริ่มต้น .NET นั้นทำการออกแบบ .DLL เสีย ซึ่งในรุ่นนี้จะมีสิ่งที่เก็บนอกจากตัว Code แล้วนั่นก็คือสิ่งที่เรียกว่า Metadata ซึ่งเป็นการอธิบายตัว .DLL ว่าภายในนั้นประกอบด้วยอะไรบ้าง เราถือ

ว่า MetaData นั้นเป็นข้อมูลที่สำคัญ ที่สามารถนำไปใช้สร้าง IntelliSense ให้กับ Visual Studio ได้ และยิ่งไปกว่านั้น MetaData ยังเก็บ Version ของ .DLL นั้นด้วย และ .DLL รุ่นใหม่นี้เองที่ Microsoft เรียกว่า Assembly

Assembly มี 2 ประเภท แบบแรกคือที่เป็น Private มันจะอยู่เฉพาะกับโปรแกรมของเรา จะไม่ไปรบกวนกับโปรแกรมอื่น และอีกแบบคือ Shared ซึ่งมันต้องไปอยู่ใน Directory ที่ใช้ร่วมกันระหว่างโปรแกรม

Assembly ที่เป็นแบบ Shared นั้น จะเก็บอยู่ในส่วนที่เรียกว่า Global Assembly Cache (GAC) ซึ่งโดยปกติแล้วจะอยู่ใน WINNT\Assembly\GAC โดยที่การที่ไม่ให้โปรแกรมทับกันนั้น ทาง Microsoft ใช้หลักการเข้ารหัสแบบ RSA โดยที่กำหนดให้บริษัททุกบริษัทที่จะนำเอา Assembly ไปเก็บไว้ใน GAC ได้นั้นต้องสร้าง Key 2 ตัว นั่นก็คือ Public Key และ Private Key ทาง Microsoft จะนำเอา 8 bytes หลังของ Public key ของบริษัทนั้นมาสร้างเป็นชื่อ Directory เช่น 1.0.2411.0__b77a5c561934e089 ส่วนแรก 1.0.2411.0 คือ Version และส่วนที่สอง b77a5c561934e089 นั้นเป็น 8 bytes สุดท้ายของ Public key ของ Microsoft ด้วยวิธีนี้จะมี การทับกันเกิดขึ้น ชื่อ Directory แบบพิเศษนี้ Microsoft เรียกว่า Strong Name (โปรแกรมที่ใช้ในการจัดการ GAC ชื่อว่า GACUtil.exe มามาให้กับ .NET Framework)

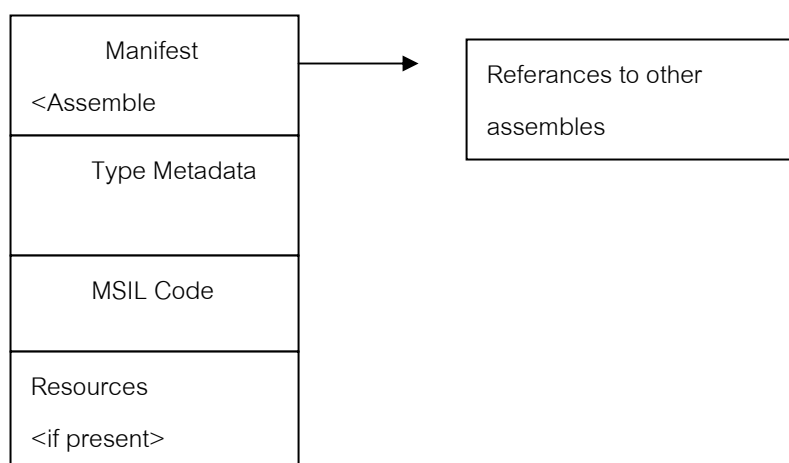
การนำ RSA มาใช้นั้นยังส่งผลถึงเรื่อง Security อีกด้วยครับกล่าวคือมันสามารถพิสูจน์ ทราบได้ว่า Assembly ตัวนี้ที่กำลังจะลงนั้น มาจากบริษัทนั้นๆ จริงหรือไม่ จะไม่มี Hacker คน ไหนสวมรอยแอบเอาโทรจันใส่เข้ามา ในอดีตนั้นเราไม่มีทางรู้ได้เลย แต่ .NET ใช้ Technology ที่ ชื่อว่า Digital Signature ทุก Assembly จะมีลายเซ็นต่ออยู่ ถ้ามีการเปลี่ยนแปลงใดๆ แม้แต่นิดเดียว มันจะทำให้ลายเซ็นตัวนั้นพัง เราจะรู้ได้เลยว่า Assembly ดังกล่าวมีปัญหา ลายเซ็นตัวนี้ปลอม ได้ยากมาก

เราจะเห็นได้ว่าโปรแกรมในอนาคตที่สร้างจาก .NET นั้นจะไม่มีปัญหาเรื่อง DLL Hell อีกต่อไป ทั้งนี้ก็เนื่องจากการใช้งานของ Fusion Technology 2.0 แต่ปัญหา DLL Hell นั้นใช้จะจบ สิ้นลงทีเดียว เรายังมี .DLL version 2 หรือ COM Component ที่ Fusion Technology 2.0 ยังไม่ รองรับ เนื่องจากใน COM Component นั้นไม่มี Meta Data ที่ Fusion Technology จะนำเอาไป สร้าง Strong Name ได้ (ส่วน DLL version 1 ที่เป็น 16 bits นั้นคงไม่ต้องกังวลมาก เพราะจำนวน โปรแกรมที่น้อยอยู่แล้ว และมันไม่สมควรที่จะรันโปรแกรมเก่าแบบนั้นแล้ว)

ทาง Microsoft มีโครงการ Fusion Technology 2.5 ที่จะมากับ Windows XP พร้อมกับ คุยไว้ว่า version 2.5 นี้คือจุดสิ้นสุดของ DLL Hell โดยถาวร โดยความสามารถที่จะจัดการกับ COM Component ใหม่ๆ ได้ด้วย

2.3.2 โครงสร้างของ .Net Assembly

ส่วนต่างๆของแอสเซมบลีได้ให้ประโยชน์ต่อโปรแกรม .NET ที่จะค้นหาเกี่ยวกับแอสเซมบลีอีกตัวหนึ่ง และแจกแจงการอ้างอิงระหว่างโปรแกรมและคอมโพเนนต์ทั้งหลาย แอสเซมบลีหนึ่งๆจะบรรจุโค้ดที่เอ็กซีคิวต์ได้สำหรับโปรแกรมหรือคลาสไลบรารี ตามด้วยเมตาดาต้า(metadata – คือข้อมูลที่อธิบายข้อมูลอีกตัวหนึ่ง) ซึ่งเปิดโอกาสให้โปรแกรมอื่นๆค้นหาคลาส เมทอด และพารามิเตอร์ทั้งหลายของออบเจกต์ที่ถูกกำหนดอยู่ในแอสเซมบลีนี้ได้ เมตาดาต้าทำงานได้สองทางคือ เป็นสารบัญที่อธิบายสิ่งต่างๆที่บรรจุอยู่ในแอสเซมบลี และเป็นบรรณานุกรมที่อธิบายการอ้างอิงข้อมูลภายนอกแอสเซมบลี โดย .NET แอสเซมบลีที่เป็นไฟล์เดี่ยวๆจะมีรูปแบบทั่วไปดังต่อไปนี้



รูปที่ 2.3 ส่วนประกอบของ Assembly

แต่ละแอสเซมบลีจะบรรจุ แมนิเฟสท์ (manifest) ซึ่งอธิบายเนื้อหาของแอสเซมบลี (เหมือนกับใบตราส่งสินค้าสำหรับการขนส่งสินค้า) แมนิเฟสท์นี้ยังเรียกได้อีกอย่างหนึ่งว่า เมตาดาต้าของแอสเซมบลี <assembly metadata> เนื่องจากมันอธิบายตัวแอสเซมบลีเองไว้มีโมดูลที่มันบรรจุอยู่บ้าง แอสเซมบลีอื่นใดที่มันอ้างอิง เหล่านี้เป็นต้น เราจะตรวจสอบสิ่งนี้ในรายละเอียดทีหลังของบทเมื่อเราไปดูเนื้อหาของแอสเซมบลีที่เราได้สร้างขึ้นมาแล้วเสร็จแล้ว เทคโนโลยีคอมโพเนนต์ก่อนหน้านี้ อาทิ COM เป็นต้น ไม่มีคอนเซปต์ที่สร้างเอาไว้ภายในอย่างเช่นแมนิเฟสท์ นี้เป็นหัวใจของการอธิบายตัวเองที่ถูกรวบรวมอยู่ใน .NET แอสเซมบลี .NETรันไทม์จะใช้แมนิเฟสท์ในแอสเซมบลีของโปรแกรมในขณะเอ็กซีคิวต์โปรแกรมสำหรับรองรับการอ้างอิงถึงแอสเซมบลีอื่นๆ ในระบบ อาทิ ไลบรารี System.Console ที่บรรจุเมธอด WriteLine<> สำหรับการพิมพ์ข้อความ “Hello ,World” ออกมาเป็นต้น แมนิเฟสท์จะต้องตามด้วย เมตาดาต้าของชนิดข้อมูล <type metadata> ซึ่งก็คือคำอธิบายของคลาสหรืออ็อปเพอตี้ เมธอด และอื่นๆ ที่บรรจุใน

แอสเซมบลีอยู่รวมกับชนิดข้อมูลสำหรับพารามิเตอร์ของมันและค่าส่งคืน แล้วตามท้ายด้วยไบนารีโค้ดจริงๆ สำหรับข้อมูลแต่ละชนิดที่ถูกจัดเก็บอยู่ในรูปของโค้ด Microsoft Intermediate Language <MSIL> ที่ไม่ขึ้นอยู่กับเครื่อง สุดท้ายยังมีรีสอร์ซ <resource> ไคยที่ประกอบกันขึ้นมาเป็นแอสเซมบลีอีกด้วย รีสอร์ซถือเป็นส่วนที่เอ็กซีคิวต์ไม่ได้ของโปรแกรมของคุณ (ระบุเอาไว้ในไฟล์ .Resources) เช่น อิมเมจ ไอคอน หรือเมสเสจไฟล์ เป็นต้น

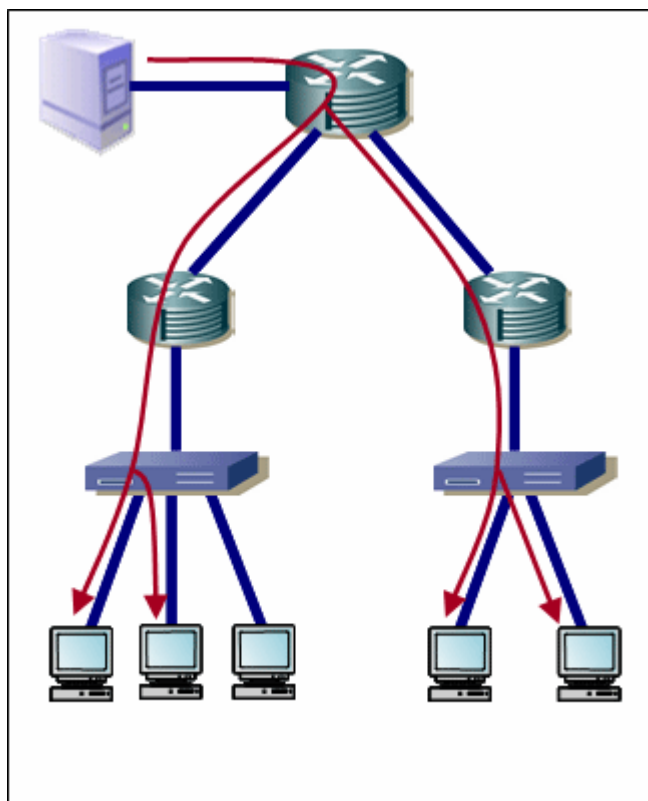
ในขณะที่แอสเซมบลีหนึ่งมักจะประกอบขึ้นมาจากไฟล์เพียงไฟล์เดียว มันก็ยังสามารถถูกสร้างขึ้นมาจากไฟล์หลายๆไฟล์ก็ได้ จากมุมมองของ .NET รันไทม์ แอสเซมบลีหลายไฟล์ (multi-file) นั่นก็คือไฟล์ DLL หรือ EXE แต่ละไฟล์ที่เกิดขึ้นมาเพื่อประกอบรวมกันเป็นหลายๆไฟล์ มีเพียงแค่ไฟล์เดียวที่บรรจุแมนิเฟสต์เอาไว้ แมนิเฟสต์จะชี้ไปยังไฟล์อื่นๆที่ทำให้เกิดแอสเซมบลีหลายไฟล์ขึ้นมา ไฟล์ที่บรรจุโค้ดที่เอ็กซีคิวต์ได้จะถูก เรียกว่าโมดูล ซึ่งไฟล์เหล่านี้บรรจุเมตาดาต้า ของชนิดข้อมูลและ MSIL โค้ด มันยังอาจเป็นรีสอร์ซไฟล์ที่ไม่มีโค้ดที่เอ็กซีคิวต์ได้บรรจุอยู่ก็ได้

แอสเซมบลีหลายไฟล์นั้นมักจำเป็นต่อแอปพลิเคชันชั้นสูงที่แน่นอนบางตัวเท่านั้น โมดูลหรือรีสอร์ซจะถูกโหลดก็ต่อเมื่อมันเอ็กซีคิวต์หรือถูกนำไปใช้จริงแล้วเท่านั้น ดังนั้นจึงสามารถประหยัดเวลาการดาวน์โหลด และหน่วยความจำได้เมื่อโมดูลและรีสอร์ซที่แทบไม่ค่อยได้ใช้ถูกจัดเก็บเอาไว้เป็นไฟล์แยกจากกัน ยกตัวอย่างเช่น แอปพลิเคชันหนึ่งที่ส่งมอบกันระหว่างประเทศ อาจมีโมดูลหรือรีสอร์ซที่เขียนขึ้นในภาษาที่แตกต่างกัน จึงควรจะแยกสิ่งเหล่านี้ออกจากกัน ดังนั้นจะมีเพียงแค่โมดูลสำหรับภาษาที่ใช้งานจริงเท่านั้นที่จะถูกโหลดเข้าไปในหน่วยความจำ

2.4 IP Multicasting

ในการส่งข้อมูลบน Internet โดยทั่วไปจะเป็นการส่งแบบ 1 ต่อ 1 กล่าวคือ มีผู้ส่ง 1 คน และผู้รับ 1 คน หรือที่เราเรียกว่า Unicast ซึ่งเมื่อจำนวนผู้รับเพิ่มขึ้นมากกว่า 1 จึงจำเป็น ต้องส่งข้อมูลซ้ำตามจำนวนของผู้รับ เช่น เมื่อต้องการส่งภาพ Video Conference ไปให้เพื่อนที่อยู่ต่าง Campus อีก 3 คน จำเป็นต้อง ส่งภาพซ้ำถึง 3 ครั้ง ซึ่งเป็นการสูญเสียเวลาและทรัพยากร รวมทั้งทำให้ข้อมูลใน Network คับคั่งได้

เทคโนโลยี Multicast เป็นเทคโนโลยีช่วยแก้ปัญหานี้ได้ โดยในการส่งแบบ 1 ต่อ N (N คือจำนวนผู้รับ) ผู้ส่งสามารถส่ง ข้อมูลเพียงครั้งเดียวให้กับผู้รับหลายคนได้ในเวลาเดียวกัน Multicast แบ่งออกเป็น 2 ประเภทคือ IP Multicast ที่ต้องใช้ ความช่วยเหลือจาก Router และ Application Layer Multicast ซึ่งเป็นเทคโนโลยีใหม่ที่ถูกพัฒนาเมื่อเร็ว ๆ นี้ ดังนั้นโดยปกติเวลาที่พูดถึง Multicast จึงหมายถึง IP Multicast



รูปที่ 2.4 แสดงการส่งข้อมูลแบบ multicast

โดยในการทำ Multicast นั้นจะทำการกำหนด IP ซึ่งเป็น Multicast Address ต่างๆเป็นช่วงๆ และแต่ละช่วงจะมีการกำหนดวัตถุประสงค์เอาไว้เฉพาะ ดังแสดงในตาราง

Range	Assignment
224.0.0.0-224.0.0.255	Local Network control block
224.0.1.0-224.0.1.255	Inter-network control block
224.0.2.0-224.0.255.0	AD-Hoc block
224.1.0.0-224.1.255.255	ST Multicast groups
224.2.0.0-224.2.255.255	SDP/SAP block
224.252.0.0-224.255.255.255	DIS Transient block
225.0.0.0-231.255.255.255	Reserved
232.0.0.0-232.255.255.255	Sourcespecific multicast block
233.0.0.0-233.255.255.255	GLOP block
234.0.0.0-238.255.255.255	Reserved
239.0.0.0-239.255.255.255	Administratively scoped block

ตารางที่ 2.1 ช่วง IP สำหรับ Multicast

โดยในช่วง IP ต่างๆที่กำหนดขึ้นมาดังในตารางข้างต้นนี้ ยังได้กำหนดลงไปอีกด้วยว่าแต่ละหมายเลข IP นั้นใช้อะไร เช่น IP Address 224.0.0.1 และ 224.0.0.2 .ในช่วงของ Local Network control block(224.0.0.0-224.0.0.255) นั้นจะเป็น IP Address ที่สงวนไว้สำหรับให้อุปกรณ์ Routers ใช้สำหรับส่งข้อมูลเกี่ยวกับ Multicast Group ระหว่างกันเท่านั้น เป็นต้น

ในปัจจุบันเทคนิคในการควบคุมการสื่อสารข้อมูลผ่านระบบเน็ตเวิร์กแบบ Multicast มีอยู่ด้วยกัน 2 วิธีคือ

1. Peer to peer เป็นวิธีซึ่งเครื่องทุกเครื่องที่ต้องการสื่อสารแบบ Multicast มีสิทธิในการรับและส่งข้อมูลเท่าๆกัน และโดยปกติแล้วด้วยเทคนิคนี้จะไม่สามารถป้องกันเครื่องภายนอกทำการลงทะเบียนเพื่อเป็นสมาชิกของ Multicast Group ได้ กล่าวคือเครื่องคอมพิวเตอร์ใดๆในซึ่งเชื่อมต่ออยู่กับ Internet สามารถสมัครเป็นสมาชิกของ Multicast Group ได้และทุกเครื่องที่เป็นสมาชิกมีสิทธิรับข้อมูลที่ Multicast ได้โดยไม่มีการกีดกันนั่นเอง

2. Central server เป็นวิธีการซึ่งใช้เครื่อง server เครื่องหนึ่งในการควบคุม Multicast Group โดยเครื่องคอมพิวเตอร์ใดที่ต้องการจะสมัครเป็นสมาชิกของ Multicast Group ต้องร้องขอสถานะภาพสมาชิกจากเครื่อง server ที่ควบคุมกลุ่มก่อน หากการร้องขอสำเร็จก็จะได้รับข้อมูล หากไม่สำเร็จก็จะไม่สามารถได้รับ Multicast packet สำหรับเทคนิคการ Multicast แบบ Central server นี้จะไม่สามารถใช้ได้บน IP Network หากแต่ในปัจจุบันจะมีเพียงโครงข่าย ATM เท่านั้นที่สนับสนุนเทคนิคการ Multicast ในรูปแบบนี้

โดยปกติอุปกรณ์ Router จะไม่ส่ง Multicast Packet ข้าม subnet เพราะฉะนั้นเพื่อให้สามารถรับ Multicast Packets ได้ต้องมีกรรมวิธีพิเศษเพิ่มเติมเข้ามา นั่นคือก่อนที่จะทำการรับ Multicast Packets เครื่องที่ต้องการรับ Multicast Packets นั้นจะต้องเข้าเป็นสมาชิกของ Multicast group ก่อน โดยในการสมัครเข้าเป็นสมาชิกของ Multicast group นั้นจะต้องมีการส่ง IGMP<Internet Group Management Protocol> ไปยังอุปกรณ์ Router ที่รับผิดชอบ subnet นั้น <local router> เพื่อทำการสมัครเข้าเป็นสมาชิกของ Multicast group ก่อน โดยในการส่ง IGMP ไปยัง Local Router นั้นก็เพื่อลงทะเบียนเครื่องคอมพิวเตอร์เครื่องนั้นไว้กับ Local Router ว่าต้องการจะรับ Multicast Packets จาก Address ใด รวมทั้งเมื่อเครื่องคอมพิวเตอร์นั้นส่ง Multicast Packets ออกไป อุปกรณ์ Router จะได้ทำการ Router Packets นั้นๆข้าม subnet เมื่อต้องการจะหยุดการรับ Multicast Packets จะต้องส่ง IGMP ไปยัง Local Router เพื่อยกเลิกการลงทะเบียนเข้าใช้บริการ Multicast group